Chapter 12: Integrating CodeQL into Development Workflows

Download this chapter as a PDF for offline reading:
Chapter-12-Integrating-CodeQL-into-Development-Workflows.pdf

A query that only runs on your laptop helps you. A query that runs on every pull request helps the whole team. This chapter shows how to put the packs and suites from Chapter 11 into continuous integration.

GitHub Actions

The simplest and most common integration is the official CodeQL Action. A minimal workflow (see `examples/chapter12/codeql-analysis.yml`):

```yaml
name: "CodeQL"

on:
  push:
    branches: [ "main" ]
  pull_request:
    branches: [ "main" ]
  schedule:
    - cron: "0 6 * * 1"

jobs:
  analyze:
    name: Analyze
    runs-on: ubuntu-latest
    permissions:
      actions: read
      contents: read
      security-events: write

    strategy:
      fail-fast: false
      matrix:
        language: [ "javascript" ]

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4

      - name: Initialize CodeQL
        uses: github/codeql-action/init@v3
        with:
          languages: ${{ matrix.language }}

      - name: Autobuild
        uses: github/codeql-action/autobuild@v3
```

```yaml
      - name: Perform CodeQL Analysis
        uses: github/codeql-action/analyze@v3
        with:
          category: "/language:${{ matrix.language }}"
```

Key points:

- The `security-events: write` permission is required so results appear in the code scanning alerts view.
- `autobuild` works for many languages; compiled languages often need a custom build step instead.
- The scheduled run (here weekly) catches newly published queries and newly introduced issues on the default branch.
- You can point `init` at a custom pack or suite with the `queries` and `packs` inputs.

Results land as code scanning alerts. Developers see them on the pull request and can dismiss or fix them before merge.

Other CI and MRVA

Other continuous-integration systems

Any CI system that can run a container or a shell can run CodeQL. The pattern is the same three steps you already know:

1. Create a database (`codeql database create`).
2. Analyze it (`codeql database analyze --format=sarif-latest`).
3. Upload the SARIF (`codeql github upload-results` or the equivalent API call).

A typical non-GitHub job therefore looks like:

```
codeql database create my-db --language=javascript --source-root=.
codeql database analyze my-db codeql/javascript-queries \
  --format=sarif-latest --output=results.sarif
codeql github upload-results \
  --repository=my-org/my-repo \
  --ref=refs/heads/main \
  --sarif=results.sarif
```

The same commands work in Jenkins, GitLab CI, Azure Pipelines, or a custom runner. The only requirement is that the runner has the CodeQL CLI and can authenticate to GitHub for the upload.

Multi-repository variant analysis (MRVA)

Once you have a query you trust, you often want to run it across many repositories at once. Multi-repository variant analysis (MRVA) lets you do exactly that. You select a query (or a suite), choose a set of repositories, and CodeQL runs the analysis in the cloud, returning a unified result set.

Typical uses:

- Confirm that a newly written query finds the issue you expect across an organization.
- Measure the impact of a candidate security rule before enabling it everywhere.
- Hunt for a specific pattern after a vulnerability disclosure.

MRVA is available in the CodeQL extension for VS Code and through the GitHub code scanning interface. It is the natural next step after a query has graduated from "works on my database" to "ready for production."

Next Steps

You can now run CodeQL on every pull request, in any CI system, and across many repositories at once. The next chapter looks at performance and debugging techniques that keep those analyses fast and the results trustworthy.

All examples referenced above live in the companion repository:
github.com/rod-trent/MustLearnCodeQL
under `examples/chapter12/`.

Must Learn CodeQL — Rod Trent
Companion repository: github.com/rod-trent/MustLearnCodeQL